

React의 State란?

컴포넌트 내부에서 관리되는 동적인 데이터이다.

컴포넌트의 상태를 나타내며, 사용자 입력 등에 따라 변경될 수 있다.

state가 변경되면 리액트는 자동으로 해당 컴포넌트와 자식 컴포넌트를 다시 렌더링 한다.

useState() 혹은 `useReducer`를 통해 state 상태를 관리할 수 있다.

리액트 훅(Hooks)

함수형 컴포넌트에서 상태와 생명주기 기능을 사용할 수 있도록 해주는 기능으로 다양한 종류의 훅이 있다. 대표적으로 `useState`, `useEffect`, `useContext`, `useReducer`, `useCallback`, `useMemo`, `useRef` 등이 있다.

State 선언

state 변수와, state 값을 변경할 함수 이름을 함께 선언하며, 우변에는 초기값을 `useState()` 의 괄호안에 작성한다. - `useState` 사용을 위해 `useState` 모듈 추가

```
import React, { useState } from 'react';
```

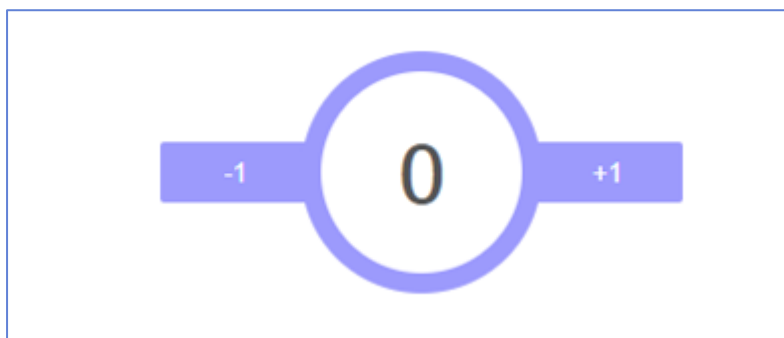
[사용법]

```
const [상태 값 저장 변수, 상태 값갱신함수] = useState(상태 초기 값);
```

[예제]

```
const [no, setNo] = useState(0);
```

useState실습



```

3 import './EX01_Count.css';
4 const Ex01_Count = () => {
5   const [no, setNo] = useState(0); //0은 no초기값 / setNo는 값변경할때 사용하는 함수
6   let i=0;
7   const minusFn = ()=>{
8     setNo(no-1);
9     i--;
10  }
11
12  const plusFn = ()=>{
13    setNo(no+1);
14    i++;
15  }
16
17  return (
18    <div className='countStyle'>
19      <button onClick={minusFn}>빼기</button>
20      <span>{no}</span> / <span>{i}</span>
21      <button onClick={plusFn}>더하기</button>
22    </div>
23  );
24 };
25
26 export default Ex01_Count;

```

전역필드 vs useState

```

step04-state > src > Ex01_Count.jsx > Ex01_Count
1 import React, { useState } from 'react';
2 import './Ex01_Count.css';
3 let i=0;
4 const Ex01_Count = () => {
5   console.log("call...")
6
7   const [ no , setNo ] = useState(0); //no변수, setNo는 함수
8
9   //더하기
10  const plusFun = ()=>{
11    i++;
12    setNo(no+1);
13    console.log(i)
14  }
15
16  //빼기
17  const minusFun = ()=>{
18    i--;
19    setNo(no-1);
20    console.log(i)
21  }
22
23  return (
24    <div className='Ex01_Count'>
25      <button onClick={minusFun}>빼기</button>
26      <span>i = {i} / no = {no}</span>
27      <button onClick={plusFun}>더하기</button>
28    </div>
29  );
30
31  export default Ex01_Count;

```

전역필드라면?

실행결과 동일하게 동작하는 것 처럼 보인다.

중요
 : 컴포넌트의 상태(state)를 리액트가 관리하고
 상태가 바뀌면 컴포넌트를 자동으로 다시 렌더링한다.

전역필드 i와 useState로 관리되는 no가 동일해보이지만
 현재 no의 값이 변경되어 화면을 다시 렌더링 하면서
 i와 no가 동일해 보인다.
 만약, no를 지우고 let i 선언한 상태에서
 i++ or i--를 해보면 화면에 다시 그려지지 않는다!

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

오후 3:39:58 [vite] (client) hmr update

State 사용하기 – Form01

EX02_Form1.jsx 생성 및 input 태그 작성

```

3  const EX02_Form1 = () => {
4    const inputName =(e)=>{
5      console.log(e.target.value)
6    }
7    return (
8      <div>
9        <h2>폼 입력값 state Test</h2>
10       이름 :<input type="text" onChange={inputName}/>
11     </div>
12   );
13 };
14
15 export default EX02_Form1;

```

name이라는 state를 정의하여 입력값을 담기

```

1  import React from 'react';
2  import { useState } from 'react';
3
4  const EX02_Form1 = () => {
5    const [name, setName] = useState("");
6
7    const inputName =(e)=>{
8      console.log(e.target.value)
9      setName(e.target.value);
10   }
11   return (
12     <div>
13       <h2>폼 입력값 state Test</h2>
14       이름 :<input type="text" value={name} onChange={inputName}/><br/>
15     </div>
16   );
17 };
18

```

나이 입력폼 추가해보자.

```

4  const EX02_Form1 = () => {
5      const [name, setName] = useState("");
6      const [age, setAge] = useState(0);
7
8      const inputName = (e) => {
9          console.log(e.target.value)
10         setName(e.target.value);
11     }
12     return (
13         <div>
14             <h2>폼 입력값 state Test</h2>
15             이름 : <input type="text" value={name} onChange={inputName}/><br/>
16             나이 : <input type="text" value={age} onChange={(e) => setAge(e.target.value)}/>
17         </div>
18     );
19 }

```

State 사용하기 -Ex03_Form02

회원정보를 입력 받는 폼을 정의하고 입력된 정보를 profile이라는 하나의 object로 state 지정

Ex03_Form2.jsx 생성 및 input 태그 작성 이름, 나이, 이메일에 각각 입력을 하면 inputUpdate 함수 호출 inputUpdate 함수는 입력한 값의 name, value를 콘솔에 출력한다.

객체 비구조화 할당 (Object Destructuring Assignment)

```
const { name, value } = e.target;
```

e.target에 저장된 name, value 속성에 담긴 값을 비구조화 할당 방식을 통해 추출

```

2 import { useState } from 'react';
3
4 const Ex03_Form2 = () => {
5   const [profile, setProfile]= useState({
6     name:"",
7     age:"",
8     email:"",
9   })
10
11   const inputUpdate =(e)=>{
12     const {name, value} = e.target; → 구조분해할당
13     console.log(name, value);
14   }
15   return (
16     <div>
17       <h2>폼 입력값 state Test - Profile</h2>
18       이름: <input type="text" name="name" value={profile.name} onChange={inputUpdate} />
19       <br></br>
20       나이: <input type="text" name="age" value={profile.age} onChange={inputUpdate} />
21       <br></br>
22       이메일: <input type="text" name="email" value={profile.email} onChange={inputUpdate} />
23     </div>
24   );
25 };
26
27 export default Ex03_Form2;

```

→ state에 Object 저장

→ 구조분해할당

state 값을 쓰지 않았기 때문에 input에 값을 입력해도 화면에 입력값이 보이지는 않는다.

State에 값 담기

inputUpdate 함수에 profile state 값을 담기 위한 코드 추가한다.

전개연산자(spread operator)를 사용하여 값을 담는다.

전개 연산자(spread operator)

전개 연산자는 배열이나 객체의 요소를 개별요소로 분리할 때 사용하는 연산자이다.

객체 값을 복사할 때 활용한다.

```

setProfile({
  ...profile,
  [name]: value,
});

```

...profile: 기존 profile에 담긴값을 그대로 가져온다.

[name]: value,; 새롭게 추가된 값은 바꿔준다.

[]를 사용하지 않으면 문자열 리터럴 'name'으로 취급되어 name이라는 속성이 추가된다.

[]사용하면 name변수의 값이 속성이름이 된다. 즉 동적으로 속성이름을 결정할수 있다.

```

11     const inputUpdate =(e)=>{
12         const {name, value} = e.target;
13         console.log(name, value);
14
15         setProfile({
16             ...profile,
17             [name] : value,
18         })
19     }
20     return (
21         <div>
22             <h2>폼 입력값 state Test - Profile</h2>
23             이름: <input type="text" name="name" value={profile.name} onChange={inputUpdate} />
24             <br></br>
25             나이: <input type="text" name="age" value={profile.age} onChange={inputUpdate} />
26             <br></br>
27             이메일: <input type="text" name="email" value={profile.email} onChange={inputUpdate} />
28         </div>
29     );
30 };

```

조건부 렌더링

state 값에 따라 화면에 보여줄 요소를 다르게 하는 것

삼항 연산자를 활용하는 것이 일반적이다.

삼항연산자

삼항연산자는 조건에 따라 실행할 내용을 구분할 수 있도록 하는 연산자로서 if, else 와 비슷한 기능을 한다. 표현이 간결하기 때문에 조건부 렌더링에 많이 활용된다.

[조건] ? 실행블록1(참인경우) : 실행블록2(거짓인경우)

- condition 변수 값에 따라 참 또는 거짓 출력

```
function ConditionalRendering1() {
  const condition = true;
  return (
    <>
      <h2>ConditionalRendering1.jsx</h2>
      {condition ? <h2>참</h2> : <h2>거짓</h2>}
    </>
  );
}
export default ConditionalRendering1;
```

조건부 렌더링 예제

조건을 따질 변수를 state로 지정하여 값이 바뀔 때 따라 화면에 보여지는 내용을 바꾼다.

버튼을 클릭할 때마다 isLogin의 반대 값을 isLogin 값으로 저장한다.

로그인상태(isLogin=true ⇒ Logout), 로그아웃상태(isLogin=false ⇒ Login) 출력한다.

```
2  import { useState } from 'react';
3
4  const Ex04_ConditionRendering = () => {
5    const [isLogin, setIsLogin] = useState(false);
6    return (
7      <div>
8        <h3>ConditionRendering Test</h3>
9        <button onClick={()=>setIsLogin(!isLogin)} >
10         {isLogin? "Logout" : "Login"}
11        </button>
12      </div>
13    );
14  };
15
16  export default Ex04_ConditionRendering;
```

참고)<https://react.dev/reference/react/useState>

Basic useState examples

1. Counter (number) 2. Text field (string) 3. Checkbox (boolean) 4. Form (two variables)

Example 4 of 4: Form (two variables)

You can declare more than one state variable in the same component. Each state variable is completely independent.

App.js

Download Reset Fork

```
1 import { useState } from 'react';
2
3 export default function Form() {
4   const [name, setName] = useState('Taylor');
5   const [age, setAge] = useState(42);
6 }
```

Taylor

Increment age

Hello, Taylor. You are 42.